KristalBASIC v1.0 User Manual

KristalBASIC is a modern, compiled programming language that combines the immense performance of the C language (zero-cost abstraction) with the easy and readable syntax of BASIC languages. This manual covers all features, data types, built-in functions, and game/graphics engine integrations in KristalBASIC v1.0.

---

1. Basic Syntax and Comments

KristalBASIC does not require semicolons (`;`) at the end of lines. Code blocks are defined using curly braces (`{ }`). Comments are marked with the # symbol:

```
# This is a comment
INT x = 10
```

---

2. Data Types and Variables

KristalBASIC is statically typed; you must explicitly define the type of a variable.

- **INT**: Integers (`INT age = 25`)
- **DECIMAL**: Floating-point numbers (`DECIMAL pi = 3.14`)
- **STRING**: Text (`STRING name = "Kristal"`)
- **BOOL**: Booleans (`BOOL active = TRUE`, `FALSE`)

String Interpolation

You can embed variables inside strings using the `${variable}` format:

```
STRING name = "Alice"
INT age = 25
print("Hello, my name is ${name} and I am ${age} years old.\n")
```

---

3. Operators

- **Mathematical**: `+`, `-`, `*`, `/`, `%`
- **Comparison**: `==`, `!=`, `<`, `>`, `<=`, `>=`
- **Logical**: `AND`, `OR`, `NOT`

---

4. Control Structures

IF / ELSE

```
IF (score > 50) {
    print("Passed\n")
} ELSE {
    print("Failed\n")
}
```

SWITCH / CASE

```
SWITCH (choice) {
    CASE 1:
        print("First\n")
    CASE 2:
        print("Second\n")
    DEFAULT:
        print("Other\n")
}
```

WHILE Loop

```
INT i = 0
WHILE (i < 10) {
    print("Number: " + i + "\n")
    i = i + 1
}
```

FOR Loop

```
FOR (INT i = 0 TO 10) {
    print("Value: " + i + "\n")
}
```

---

5. Dynamic Data Structures

LIST (Dynamic Arrays)

Lists can flexibly store multiple items of the same type.

```
LIST INT numbers = NEW LIST
numbers.append(10)
numbers.append(20)

print("Item count: " + numbers.length() + "\n")
print("First item: " + numbers.get(0) + "\n")

# Iterate with FOR IN
FOR (INT n IN numbers) {
    print("Number: " + n + "\n")
}
```

DICT (Hash Maps / Dictionaries)

Stores Key-Value pairs.

```
DICT STRING INT ages = NEW DICT
ages.put("Alice", 25)
ages.put("Bob", 30)

IF (ages.has("Alice")) {
    print("Alice's age: " + ages.get("Alice") + "\n")
}

ages.remove("Alice")
print("Dictionary size: " + ages.length() + "\n")
```

---

6. Advanced Types (OOP and Structs)

ENUM (Enumerations)

Zero-cost enumerations:

```
ENUM State {
    IDLE,
    STARTED = 10,
    FINISHED
}

INT current = State.STARTED
```

STRUCT (Data Structures)

Memory-friendly data packets, exactly like in C:

```
STRUCT Vector2 {
    DECIMAL x
    DECIMAL y
}

Vector2 v = NEW Vector2
v.x = 10.5
v.y = 20.0
```

CLASS (Classes)

Object-Oriented Programming (OOP) support:

```
CLASS Player {
    STRING name
    INT hp

    FUNC VOID TakeDamage(INT amount) {
        THIS.hp = THIS.hp - amount
    }
}

Player p1 = NEW Player
p1.name = "Hero"
p1.hp = 100
p1.TakeDamage(20)

# Manually free memory if the object is no longer needed
DELETE p1
```

---

7. Functions and Modules

Writing Custom Functions

```
FUNC INT Add(INT a, INT b) {
    RETURN a + b
}
```

IMPORT (Module Loading)

Importing code from different files:

```
# Loads utils.kbas with the alias 'u'
IMPORT "utils.kbas" AS u

INT result = u.Add(5, 10)
```

---

8. Error Handling (Try-Catch)

```
TRY {
    # Risky code
    IF (x == 0) {
        THROW "Division by zero error!"
    }
} CATCH (err_msg) {
    print("ERROR: " + err_msg + "\n")
}
```

---

9. String Methods

- `name.length()`: Returns character count (`INT`).
- `name.toUpper()`: Converts to uppercase.
- `name.toLower()`: Converts to lowercase.
- `name.substring(start, length)`: Extracts a portion of the string.
- `name.replace(old, new)`: Replaces text.
- `name.trim()`: Removes leading and trailing whitespace.
- `name.indexOf(search)`: Searches for text (Returns `-1` if not found).
- `name.split(separator)`: Splits the string and returns a `LIST_STRING`.

---

10. File (I/O) Operations

- `WRITE_FILE("log.txt", "Hello\n")`: Creates and writes to a file.

- `APPEND_FILE("log.txt", "Appended line\n")`: Appends to a file.
- `READ_FILE("log.txt")`: Reads file content as a `STRING`.
- `FILE_EXISTS("log.txt")`: Checks if file exists (`BOOL`).
- `FILE_SIZE("log.txt")`: Gets file size in bytes (`INT`).
- `DELETE_FILE("log.txt")`: Deletes the file (`BOOL`).

---

11. Console Operations

- `print("Message")`: Prints output. (Use `\n` for a new line).
- `INPUT()`: Waits for user input as a `STRING`.

---

12. Math Functions

- `MIN(a, b)`, `MAX(a, b)`: Minimum and maximum values.
- `ABS(x)`: Absolute value.
- `SQRT(x)`: Square root.
- `POW(base, exp)`: Power / Exponentiation.
- `FLOOR(x)`, `CEIL(x)`: Round down/up.
- `SIN(x)`, `COS(x)`, `TAN(x)`: Trigonometric functions (Radians).
- `ASIN(x)`, `ACOS(x)`, `ATAN2(y, x)`: Inverse trigonometric functions.
- `RAD(degree)`, `DEG(radian)`: Radian/Degree conversion.
- `CLAMP(value, min, max)`: Clamps value within a range.
- `LERP(start, end, amount)`: Linear interpolation.
- `RAND(min, max)`: Generates a random integer within range.

---

13. Graphics and Game Engine (Raylib Integration)

Window Management

- `INIT_WINDOW(width, height, "Title")`: Creates a window.
- `SET_FPS(60)`: Locks framerate.
- `WINDOW_SHOULD_CLOSE()`: Was the close button pressed? (`BOOL`).
- `TOGGLE_FULLSCREEN()`: Toggles full-screen mode.

- `CLOSE_WINDOW()`: Closes the window.
- `CLEAR_BACKGROUND(HexColor)`: Clears background (e.g., `0x181818`).
- `BEGIN_FPS() / DRAW_FPS(x, y)`: FPS display.
- `GET_TIME()`: Time elapsed in seconds (`DECIMAL`).
- `FRAME_TIME()`: Time since last frame (DeltaTime) (`DECIMAL`).

Input (Keyboard & Mouse)

- `KEY_DOWN(key)`, `KEY_PRESSED(key)`, `KEY_RELEASED(key)`: Keyboard controls (e.g., `'W'`).
- `MOUSE_DOWN(btn)`, `MOUSE_PRESSED(btn)`: Mouse buttons (0: Left, 1: Right, 2: Middle).
- `MOUSE_X()`, `MOUSE_Y()`: Mouse coordinates on screen (`INT`).
- `MOUSE_DELTA_X()`, `MOUSE_DELTA_Y()`: Mouse movement since last frame (`DECIMAL`).
- `MOUSE_WHEEL()`: Mouse wheel movement (`DECIMAL`).
- `HIDE_CURSOR()`, `SHOW_CURSOR()`, `DISABLE_CURSOR()`: Cursor settings.

2D Drawing

- `DRAW_CIRCLE(x, y, radius, HexColor)`
- `DRAW_RECTANGLE(x, y, width, height, HexColor)`
- `DRAW_LINE(x1, y1, x2, y2, HexColor)`
- `DRAW_TEXT("Text", x, y, size, HexColor)`

3D Drawing and Vectors

Vector 3D structures use the built-in `VEC3` object to boost performance.

- `VEC3(x, y, z)`: Defines a new vector.
- `VEC3_LEN(v)`: Vector length.
- `VEC3_NORM(v)`: Normalized vector.
- `VEC3_DOT(v1, v2)`: Dot product.

Entering the 3D World:

- `BEGIN_3D(cam_x, cam_y, cam_z, target_x, target_y, target_z)`: Starts 3D drawing mode.
- `END_3D()`: Ends 3D mode.

3D Drawing Functions:

- `DRAW_CUBE(v_pos, width, height, depth, HexColor)`
- `DRAW_CUBE_WIRES(v_pos, width, height, depth, HexColor)`
- `DRAW_SPHERE(v_pos, radius, HexColor)`
- `DRAW_SPHERE_WIRES(v_pos, radius, HexColor)`
- `DRAW_PLANE(v_pos, width, depth, HexColor)`
- `DRAW_GRID(slices, spacing)`: Draws a floor grid.
- `DRAW_CYLINDER(v_pos, top_radius, bottom_radius, height, slices, HexColor)`
- `DRAW_CAPSULE(v_start, v_end, radius, slices, rings, HexColor)`
- `DRAW_LINE3D(v_start, v_end, HexColor)`

Procedural Terrain System

- `DRAW_TERRAIN(center_x, center_z, radius, amplitude, HexColor)`: Draws 3D Perlin Noise-based hilly terrain.
- `TERRAIN_HEIGHT(x, z, amplitude)`: Returns the terrain height at a given (x, z) coordinate, useful for making the player walk on ground (`DECIMAL`).

End of KristalBASIC v1.0 User Manual. *Developed with love by Kristalsoft in Turkey.*